LAYER 13 OF 13

AVAILABILITY RECOVERY

Making Sure Your App Stays Up — and Bounces Back When It Doesn't

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 13: Availability & Recovery—the final layer of the 13-Layer Tech Stack. This is about making sure your app is always available for your users, and when something does go catastrophically wrong (and eventually it will), you can bring it back quickly. Think of it as the emergency plan for your entire tech stack.

You're not going to write disaster recovery scripts by hand. You're going to describe your availability requirements to your AI coding tool and let it set up monitoring, backups, and recovery procedures. Your job is to understand what uptime monitoring does (watches your app and alerts you when it goes down), what backup strategies protect you (copies of your data stored safely), and what a recovery plan looks like (step-by-step instructions for getting back online). That's what this certification proves.

This kit walks you through three levels. Tier 1 is for a solo builder who needs basic uptime monitoring and backups so they don't lose everything. Tier 2 is for a growing product that needs automated failover and tested recovery procedures. Tier 3 is for enterprise platforms with strict SLA commitments, multi-region redundancy, and formal incident response. Start at Tier 1—this is the capstone of everything you've built across all 13 layers.

Why It Matters

Everything you've built across the other 12 layers—your frontend, your backend, your database, your authentication, your payments—none of it matters if your app is down. Availability is the foundation that everything else sits on. A feature that doesn't load is worse than a feature that doesn't exist, because it erodes trust every time it fails.

Here's what most vibecoders don't think about until it's too late: your database crashes and you realize you have no backup. Your hosting provider has an outage and your app is offline for eight hours. A bad deployment takes down your entire site and you don't know how to roll back. These aren't hypothetical scenarios—they happen to real apps every day. This kit teaches you how to prepare for them so that when the worst happens, you're ready to bounce back in minutes, not days.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here’s a quick look at who each tier is for and what you’ll prove you can do.

Tier

Who It’s For

What You Prove

Tier 1

Solo builders who need their app to stay up and their data to survive disasters

You understand uptime monitoring, have basic backups in place, and know how to recover your app when something goes wrong.

Tier 2

Growing teams with apps that need automated recovery and tested failover

You can implement automated failover, run recovery drills, maintain runbooks, and design systems that recover from failures without manual intervention.

Tier 3

Platforms with SLA commitments and zero-tolerance for extended outages

You can architect multi-region redundancy, define and meet SLA targets, run chaos engineering tests, and manage a formal incident response process.

TIER 1 CERTIFICATION GOAL

You understand why availability matters, have basic uptime monitoring and backups set up, and know how to bring your app back online when something goes wrong.

What You Need to Know

You don't need to build recovery systems from scratch. You need to understand what can go wrong, what protects you, and how to get back online when disaster strikes.

Uptime monitoring (is your app alive?): Uptime monitoring is a service that checks your app every minute (or every few seconds) to see if it responds. If your app goes down, you get a notification immediately—via text, email, or Slack. Without monitoring, you find out your app is down when a customer tells you. Services like UptimeRobot, Better Uptime, and Vercel's built-in checks handle this for free.

Health checks: A health check is a special URL in your app (usually /health or /api/health) that returns a simple "OK" when everything is working. Your monitoring service hits this URL regularly. If it stops returning "OK," the monitoring service knows something is broken and alerts you. Think of it as a heartbeat monitor for your app.

Backup basics: A backup is a copy of your data stored separately from your main database. If your database crashes, gets corrupted, or gets accidentally deleted, you restore from the backup. The two critical questions are: how often are backups created (every hour? every day?) and where are they stored (a different server, a different cloud provider, a different continent)?

Recovery time (how fast can you bounce back?): Recovery time is how long it takes to get your app back online after a failure. If your database crashes and you have a daily backup, your worst-case data loss is 24 hours of data. If you have hourly backups, it's one hour. Know your recovery time so you can set realistic expectations with your users.

The vibecoder availability workflow: Tell AI to set up uptime monitoring → AI configures a health check endpoint and connects a monitoring service → set up automated database backups → test your backup by restoring from it at least once → document your recovery steps so you know what to do when something breaks → sleep better.

Your Toolkit

These tools keep watch when you can't and protect your data when things go wrong.

UptimeRobot or Better Uptime (free tier): Checks your app every minute and sends you a notification the moment it goes down. Set it up once and forget about it—it does the watching for you.

Your platform's built-in backups: Vercel, Railway, Supabase, and other platforms include automated database backups. Check that they're turned on, check how often they run, and verify you know how to restore from one.

A recovery document: A simple document (even a Google Doc) that lists exactly what to do when your app goes down: who to contact, how to check logs, how to restore from a backup, how to redeploy. Write it while you're calm so you have it when you're panicking.

Certification Exam Topics

Every exam question is scenario-based. You'll face real situations and need to identify the right response. Here's what gets tested:

Monitoring gap: Your app has been down for three hours and you didn't know until a customer emailed you. What should you have set up to find out immediately?

Health check purpose: Your monitoring service says your app is "up" but users say pages won't load. Your health check only checks if the server responds. What's missing from the health check?

Backup verification: You set up daily database backups six months ago. Have you ever tested restoring from one? What's the risk if you haven't?

Recovery time: Your database crashes and your most recent backup is 23 hours old. How much data have you lost, and what would you change to reduce that loss?

Deployment rollback: You deployed a new version and now the app is broken. What's the fastest way to get the previous working version back online?

Data loss scenario: You accidentally deleted a critical database table. You have no backups. What are your options—and how do you prevent this from happening again?

Monitoring setup: You want to be notified within two minutes if your app goes down. What do you tell AI to set up?

Backup storage: Your database backup is stored on the same server as your database. Why is this a problem and where should backups be stored instead?

Common Pitfalls

These are the mistakes vibecoders make most often with availability and recovery. They're the kind of mistakes you don't notice until the worst possible moment. Fix them while things are calm.

Not having uptime monitoring. Your app has been down for hours and you're the last person to know. A free monitoring tool would have texted you in sixty seconds.

Having backups but never testing them. A backup you've never restored from is a backup you hope works. Hope is not a recovery strategy. Test your restore process at least once.

Storing backups in the same place as your data. If the server dies, your backup dies with it. Backups belong on a different server, in a different region, ideally with a different provider.

No rollback plan for deployments. You ship a bad update and your only option is to fix it while the app is broken. A one-click rollback to the previous version gets you back online in seconds.

Waiting for a real outage to test your recovery process. If the first time you follow your runbook is during a real emergency, you'll discover its gaps at the worst possible time. Run drills.

Not communicating with users during outages. Silence makes users think you don't know or don't care. A status page that says "we know and we're working on it" builds trust even during failures.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Do you have uptime monitoring that notifies you within minutes when your app goes down?

Do you have automated database backups running—and do you know how often they run and where they're stored?

Have you ever actually restored from a backup to prove it works?

Do you have a written recovery plan—even a simple one—that lists what to do when your app goes down?

If you deployed a bad update right now, could you roll back to the previous version in under five minutes?

Do you know how much data you'd lose if your database crashed right now (based on your backup frequency)?

If your app went down during business hours, do you have a way to communicate with affected users (a status page, email, or social media plan)?

AI Audit Prompt Template

```
Copy this prompt into your AI coding tool to get a quick health check on your availability and recovery setup. It checks the same things the certification exam covers.
Review my app's availability and recovery setup and check the following. For each one, tell me pass or fail with a specific example:
Uptime monitoring: Is a monitoring service actively checking my app and alerting me when it goes down, or could my app be offline right now without anyone knowing?
Health checks: Does my app have a health check endpoint that verifies the app and its dependencies (database, external APIs) are actually working—not just that the server responds?
Database backups: Are automated backups running on a regular schedule, stored in a different location from my primary database?
Backup testing: Has a backup been successfully restored at least once to verify the backup process actually works?
Deployment rollback: Can I quickly roll back to the previous version of my app if a deployment breaks something?
Recovery documentation: Is there a written recovery plan or runbook that lists step-by-step instructions for common failure scenarios?
Give me an overall score out of 6 and list the top 3 things to fix first.
What’s Next
Congratulations—you've reached the final layer of the 13-Layer Tech Stack. If you've worked through all 13 layers, you now have a complete understanding of everything that makes a production app work: from the frontend your users see to the recovery plan that keeps you online when things go wrong. That's a real accomplishment.
The best way to prepare for this exam: audit your own app. Is uptime monitoring set up? Are backups running? Have you ever restored from one? Do you have a recovery plan written down? Every gap you find and fix is exactly what the exam tests. And once you pass, you'll have proven you can not only build with AI—you can build things that stay up and bounce back.
```

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai